

Correction — Baccalauréat NSI 2026

Sujet 26-NSIJ1G11

Exercice 1 — Algorithmes de tri et arbre binaire de recherche (6 points)

Partie A : Dictionnaire et tri

Question 1. Calcul du score d'Alex

Formules de calcul :

- 100 m : $(20 - 13,0) \times 10 = 70$ points
- Saut en longueur : $5,2 \times 20 = 104$ points
- Lancer de poids : $9,0 \times 10 = 90$ points
- 1500 m : $(500 - 310,0) \times 1 = 190$ points

Score total d'Alex = 70 + 104 + 90 + 190 = 454 points

Question 2. Compléter la fonction `nb_points` (lignes 6, 8, 10, 11)

```
def nb_points(epreuve, valeur):
    points = 0
    if epreuve == '100m':
        points = (20 - valeur) * 10
    elif epreuve == 'longueur':          # ligne 6
        points = valeur * 20
    elif epreuve == 'poids':             # ligne 8
        points = valeur * 10
    elif epreuve == '1500m':             # ligne 10
        points = (500 - valeur) * 1
    return points                        # ligne 11
```

Question 3. Compléter la fonction `score` (lignes 3, 5, 6)

```
def score(athlete):
    total = 0
    performances = athlete['performances']          # ligne 3
    for epreuve in performances:
        valeur = performances[epreuve]              # ligne 5
        total += nb_points(epreuve, valeur)         # ligne 6
    athlete['score'] = total
```

Question 4. Compléter la ligne 6 de la fonction `classer`

```
def classer(l):
    n = len(l)
    for i in range(n):
        max_index = i
        for j in range(i + 1, n):
            if l[j]['score'] > l[max_index]['score']:  # ligne 6
                max_index = j
        temp = l[i]
        l[i] = l[max_index]
        l[max_index] = temp
```

Question 5. Type de tri effectué par `classer`

Il s'agit d'un **tri par sélection**. À chaque itération de la boucle externe, on recherche le maximum parmi les éléments restants et on le place à la bonne position.

Question 6. Coût de la fonction classer

Pour une liste de taille n , la boucle externe s'exécute n fois et la boucle interne effectuée respectivement $n-1, n-2, \dots, 1$ comparaisons.

Nombre total de comparaisons = $(n-1) + (n-2) + \dots + 1 = \mathbf{n(n-1)/2}$

Le coût est donc **$O(n^2)$** (quadratique), quelle que soit la configuration initiale.

Partie B : Arbre binaire de recherche

Question 7. Créer l'objet alex

```
alex = Athlete('Alex', 13.0, 5.2, 9.0, 310.0)
```

Question 8. Écrire la méthode calculer_score

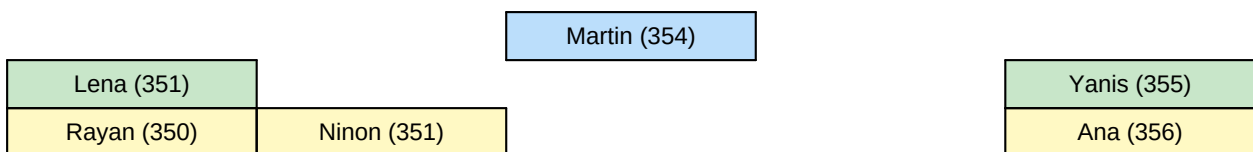
```
def calculer_score(self):
    points_100m = (20 - self.m100) * 10
    points_longueur = self.longueur * 20
    points_poids = self.poids * 10
    points_1500m = (500 - self.m1500) * 1
    return points_100m + points_longueur + points_poids + points_1500m
```

Question 9. Dessiner l'arbre obtenu après insertion des six athlètes

Ordre d'insertion et scores : Martin(354), Lena(351), Rayan(350), Yanis(355), Ninon(351), Ana(356)

Construction pas à pas :

- Insertion Martin(354) → racine
- Insertion Lena(351) → $351 < 354$ → gauche de Martin
- Insertion Rayan(350) → $350 < 354$ → gauche ; $350 < 351$ → gauche de Lena
- Insertion Yanis(355) → $355 \geq 354$ → droite de Martin
- Insertion Ninon(351) → $351 < 354$ → gauche ; $351 \geq 351$ → droite de Lena
- Insertion Ana(356) → $356 \geq 354$ → droite ; $356 \geq 355$ → droite de Yanis



Structure : Martin est la racine. Lena est son fils gauche (Rayan à sa gauche, Ninon à sa droite). Yanis est le fils droit de Martin (Ana à sa droite).

Question 10. Compléter la méthode inserer

```
def inserer(self, athlete):
    if athlete.score < self.valeur.score:
        if self.gauche == None: # ligne 9
            self.gauche = Noeud(athlete)
        else:
            self.gauche.inserer(athlete) # ligne 12
    else:
        if self.droite == None: # ligne 14
            self.droite = Noeud(athlete)
        else:
            self.droite.inserer(athlete) # ligne 17
```

Question 11. Type de parcours de la méthode classer

Il s'agit d'un **parcours infixe (in-order)** mais exploré dans l'ordre **droite** → **racine** → **gauche** (ordre infixe inversé).

La méthode visite d'abord le sous-arbre droit (scores plus grands), puis ajoute le nœud courant, puis visite le sous-arbre gauche (scores plus petits). Cela produit une liste triée du score le plus grand au plus petit, ce qui correspond au classement décroissant souhaité.

Question 12. Avantages et inconvénients des deux approches

Approche	Avantage	Inconvénient
Dictionnaires + tri	Simple à implémenter et comprendre.	Trifacile à adapter pour de grandes listes. Le tri recrée tout le tableau.
Arbre binaire de recherche (BDD)	Recherche et classement efficaces : $O(\log n)$	Plus complexe à implémenter. Restitue une liste générée à l'ins.

Exercice 2 — Chiffrement de Polybe (6 points)

Grille de référence (exercice) :

	1	2	3	4	5	6
1	Q	7	A	X	2	J
2	9	E	H	0	R	M
3	L	Z	4	W	D	O
4	6	V	N	B	8	K
5	P	Y	1	S	T	F
6	G	C	3	I	U	5

Question 1. Déchiffrer (6,2) (3,6) (3,5) (2,2)

On lit la grille ligne par colonne :

- (6,2) → ligne 6, colonne 2 → **C**
- (3,6) → ligne 3, colonne 6 → **O**
- (3,5) → ligne 3, colonne 5 → **D**
- (2,2) → ligne 2, colonne 2 → **E**

Message déchiffré : **CODE**

Question 2. Chiffrer BAC avec la clé SECURITY1024

Ordre d'insertion avec la clé SECURITY1024 :

SECURITY1024ABDFGHJKLMNPOQVWXYZ356789

La grille est :

	1	2	3	4	5	6
1	S	E	C	U	R	I
2	T	Y	1	0	2	4
3	A	B	D	F	G	H
4	J	K	L	M	N	O
5	P	Q	V	W	X	Z
6	3	5	6	7	8	9

- B → ligne 3, colonne 2 → **(3,2)**
- A → ligne 3, colonne 1 → **(3,1)**
- C → ligne 1, colonne 3 → **(1,3)**

Message chiffré : **(3,2) (3,1) (1,3)**

Question 3. Pourquoi le chiffrement de Polybe est-il symétrique ?

Le chiffrement est **symétrique** car la même clé (et donc la même grille) sert à la fois à chiffrer et à déchiffrer. Il n'y a pas de clé publique et de clé privée distinctes : quiconque possède la clé peut indifféremment chiffrer ou déchiffrer.

Question 4. Résultat de generer_ordre('AXU7')

La fonction part de 'AXU7' puis ajoute toutes les lettres/chiffres absents dans l'ordre de la chaîne de référence :

```
'AXU7BCDEFGHIJKLMNOPQRSTUVWXYZ0123456789'
```

(A, X, U, 7 déjà présents → ignorés lors du parcours de l'alphabet)

Question 5. Coder la fonction grille_vide(n)

```
def grille_vide(n):  
    return [[' ' for _ in range(n)] for _ in range(n)]
```

Question 6. Compléter generer_grille (lignes 5, 6, 7)

```
def generer_grille(cle):  
    ordre_insertion = generer_ordre(cle)  
    grille = grille_vide(6)  
    indice = 0  
    for i in range(6):  
        # ligne 5  
        for j in range(6):  
            # ligne 6  
            grille[i][j] = ordre_insertion[indice] # ligne 7  
            indice = indice + 1  
    return grille
```

Question 7. Compléter dechiffrer (ligne 5)

```
def dechiffrer(cle, message):  
    resultat = ''  
    grille = generer_grille(cle)  
    for t in message:  
        resultat = resultat + grille[t[0]-1][t[1]-1] # ligne 5  
    return resultat
```

Les indices du tuple sont 1-indexés dans l'énoncé, d'où le -1 pour accéder à la liste Python (0-indexée).

Question 8. Écrire la fonction generer_dico

```
def generer_dico(cle):  
    grille = generer_grille(cle)  
    dico = {}  
    for i in range(6):  
        for j in range(6):  
            caractere = grille[i][j]  
            dico[caractere] = (i + 1, j + 1) # positions 1-indexées  
    return dico
```

Question 9. Écrire la fonction chiffrer

```
def chiffrer(cle, message):  
    dico = generer_dico(cle)  
    resultat = []  
    for caractere in message:  
        resultat.append(dico[caractere])  
    return resultat
```

Question 10. Différence entre chiffrement symétrique et asymétrique

Chiffrement symétrique : la même clé est utilisée pour chiffrer et déchiffrer. Il est rapide mais nécessite un échange sécurisé de la clé entre les deux parties avant toute communication (ex. : AES, chiffrement de Polybe).

Chiffrement asymétrique : on utilise une paire de clés : une clé publique (connue de tous, sert à chiffrer) et une clé privée (connue du seul destinataire, sert à déchiffrer). Il résout le problème de l'échange de clé

mais est plus lent (ex. : RSA).

Alice et Bob ont raison de vouloir utiliser un algorithme asymétrique pour s'échanger leur clé Polybe quotidienne : cela évite de devoir se retrouver physiquement ou d'utiliser un canal non sécurisé.

Exercice 3 — Démineur : POO, récursivité et bases de données (8 points)

Partie A : La classe Demineur

Question 1. Compléter l'assertion (ligne 7)

```
assert 0.10 <= pourcentage_mines <= 0.30, \
    'Le pourcentage de mines doit être compris entre 10% et 30%.'
```

Question 2. Compléter les lignes 8, 9 et 10 (attributs d'instance)

```
self.hauteur          = hauteur          # ligne 8
self.largeur          = largeur          # ligne 9
self.pourcentage_mines = pourcentage_mines # ligne 10
```

Question 3. Créer l'instance demineur_intermediaire

```
demineur_intermediaire = Demineur(16, 16, 0.156)
```

Partie B : Création de la grille

Question 4. Compléter grille_demineur_vide

```
def grille_demineur_vide(self):
    return [[0 for _ in range(self.largeur)] for _ in range(self.hauteur)]
```

Question 5. Compléter placer_mines (lignes 14 à 17)

```
def placer_mines(self):
    compteur_mines = 0
    nombre_bombes = self.largeur * self.hauteur * self.pourcentage_mines
    while compteur_mines < nombre_bombes:
        ligne = randint(0, self.hauteur - 1)
        colonne = randint(0, self.largeur - 1)
        if self.grille_demineur[ligne][colonne] == 0:
            self.grille_demineur[ligne][colonne] = -1
            compteur_mines = compteur_mines + 1
```

Question 6. Écrire nombre_voisines_avec_mines

```
def nombre_voisines_avec_mines(self, coordonnees_case):
    liste_voisines = self.voisines(coordonnees_case)
    compteur = 0
    for voisine in liste_voisines:
        ligne, colonne = voisine
        if self.grille_demineur[ligne][colonne] == -1:
            compteur += 1
    return compteur
```

Question 7. Écrire generer_demineur

```
def generer_demineur(self):
    for ligne in range(self.hauteur):
        for colonne in range(self.largeur):
            if self.grille_demineur[ligne][colonne] != -1:
                self.grille_demineur[ligne][colonne] = \
                    self.nombre_voisines_avec_mines((ligne, colonne))
```

Partie C : Interface utilisateur

Question 8. Écrire la méthode visibilité (avec récursivité)

Principe : si la case choisie contient une mine (-1), on dévoile toute la grille. Sinon, on dévoile la case courante. Si elle vaut 0 (aucune mine voisine), on appelle récursivement la méthode sur toutes ses voisines encore cachées.

```
def visibilite(self, coordonnees_case):
    ligne, colonne = coordonnees_case
    valeur = self.grille_demineur[ligne][colonne]

    # Cas mine : on dévoile toute la grille
    if valeur == -1:
        for i in range(self.hauteur):
            for j in range(self.largeur):
                self.grille_visibilite[i][j] = True
        return

    # On dévoile la case courante
    if self.grille_visibilite[ligne][colonne]:
        return # déjà visible, on arrête la récursion
    self.grille_visibilite[ligne][colonne] = True

    # Si la case est 0 (aucune mine voisine), on propage
    if valeur == 0:
        for voisine in self.voisines(coordonnees_case):
            self.visibilite(voisine)
```

Partie D : Jouer en ligne — SQL

Question 9. Clés étrangères de la table Meilleur_score

La table **Meilleur_score** possède deux clés étrangères :

- **joueur** fait référence à la clé primaire **id_joueur** de la table **Joueur**.
- **niveau** fait référence à la clé primaire **niveau** de la table **Demineur**.

Question 10. Requête pour obtenir le tableau niveau / meilleur score

```
SELECT niveau, score
FROM Meilleur_score
ORDER BY score DESC;
```

Si on veut uniquement le meilleur score par niveau (en éliminant les doublons) :

```
SELECT niveau, MAX(score) AS score
FROM Meilleur_score
GROUP BY niveau
ORDER BY score DESC;
```

Question 11. Mise à jour du mot de passe de Kirna

```
UPDATE Joueur
SET mot_de_passe = 'cGhhxDE4'
WHERE pseudo = 'Kirna';
```

Question 12. Résultat de la requête JOIN

La requête sélectionne les pseudos des joueurs ayant un score au niveau 'expert' supérieur à 1000 et un temps inférieur à 400.

Dans la table **Meilleur_score**, seule la ligne joueur=2, niveau='expert', score=1196, temps=366 satisfait toutes les conditions (366 < 400 et 1196 > 1000). Le joueur d'id 2 est **Raptor**.

Résultat : **Raptor**

Question 13. Corriger 'facile' en 'débutant' dans la table Demineur

Il faut d'abord mettre à jour les références dans Meilleur_score, puis la table Demineur :

```
UPDATE Meilleur_score  
SET niveau = 'debutant'  
WHERE niveau = 'facile';
```

```
UPDATE Demineur  
SET niveau = 'debutant'  
WHERE niveau = 'facile';
```